CpT_S 440 Project Report:
Sentiment Analysis of Text Data

Fine-Tuning BERTweet on Sentiment140 Dataset

Team 7: Brian Leung, Carlos Munoz-Salazar, Cayden Calo

1. Introduction

Sentiment analysis aims to classify text into categories such as positive or negative. This task is particularly challenging for social media data due to informal language, abbreviations, and noise.

In this project, we perform sentiment classification on Twitter data using BERTweet, a Transformer model pretrained on large-scale tweet corpora. We systematically compare full fine-tuning with parameter-efficient fine-tuning (PEFT) methods, including LoRA and prompt tuning, to understand the trade-offs between performance, efficiency, and training cost.

2. Problem Formulation

Given a tweet x, we predict a binary sentiment label:

$$y \in \{0,1\}$$

where:

- 0 = negative
- 1 = positive

The model outputs:

$$P(y \mid x) = \text{softmax}(f_\theta(x))$$

and is trained using standard cross-entropy loss.

3. Dataset and Preprocessing

We use the Sentiment140 dataset, consisting of 1.6 million labeled tweets.

Preprocessing Pipeline

- Removed unused columns: ID, Date, Flag, User

- Filtered labels to {0, 4} and remapped to {0, 1}
- Normalized text:
    - User mentions → @USER
    - URLs → HTTPURL
- Encoding: ISO-8859-1 (Latin-1)
- Data split:
    - 90% training
    - 5% validation
    - 5% test
    - Random seeds: 1, 2, 3

4. Model and Methods

4.1 Base Model: BERTweet

We use vinai/bertweet-base with a 2-class classification head. This model is specifically pretrained on Twitter data, making it well-suited for this task.

4.2 Common Training Setup

Across all methods:

- Optimizer: AdamW
- Batch size:
    - Train: 16 (Effective batch size = 32)
    - Eval: 64
- Gradient accumulation: 2
- Epochs: 3
- Evaluation: per epoch
- Model selection: best validation F1
- Mixed precision: fp16 enabled

4.3 Fine-Tuning Strategies

(1) Full Fine-Tuning – Upper Bound

- All model parameters are trainable
- Learning rate: $2e-5$
- Warmup ratio: 0.06
- Weight decay: 0.01

This serves as the upper bound in performance.

(2) Classifier Head Only – Lower Bound

- Backbone (model.roberta) is frozen
- Only classification head is trained
- Same hyperparameters as full fine-tuning

This serves as the lower bound, testing how much performance relies on representation learning.

(3) LoRA (Low-Rank Adaptation)

- Injected into attention layers (query, value)
- Rank $r = 4, 8, 32$
    - LoRA trained on various rank values
- $\alpha = 32$, dropout = 0.1
- Backbone weights frozen

Training-specific choices:

- Learning rate: $2e-4$
- Fixed max sequence length: 128

(4) Prompt Tuning

- Adds 20 virtual tokens as soft prompts
- Initialized using task-specific text:

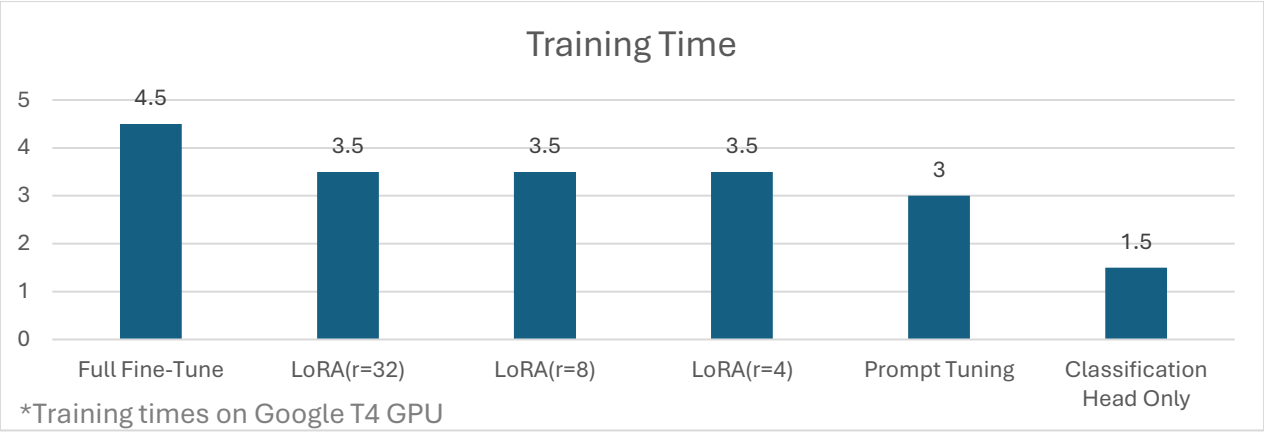
    - "Classify the sentiment of this tweet as positive or negative:"
- Backbone frozen
- Classification head also unfrozen

Training-specific choices:

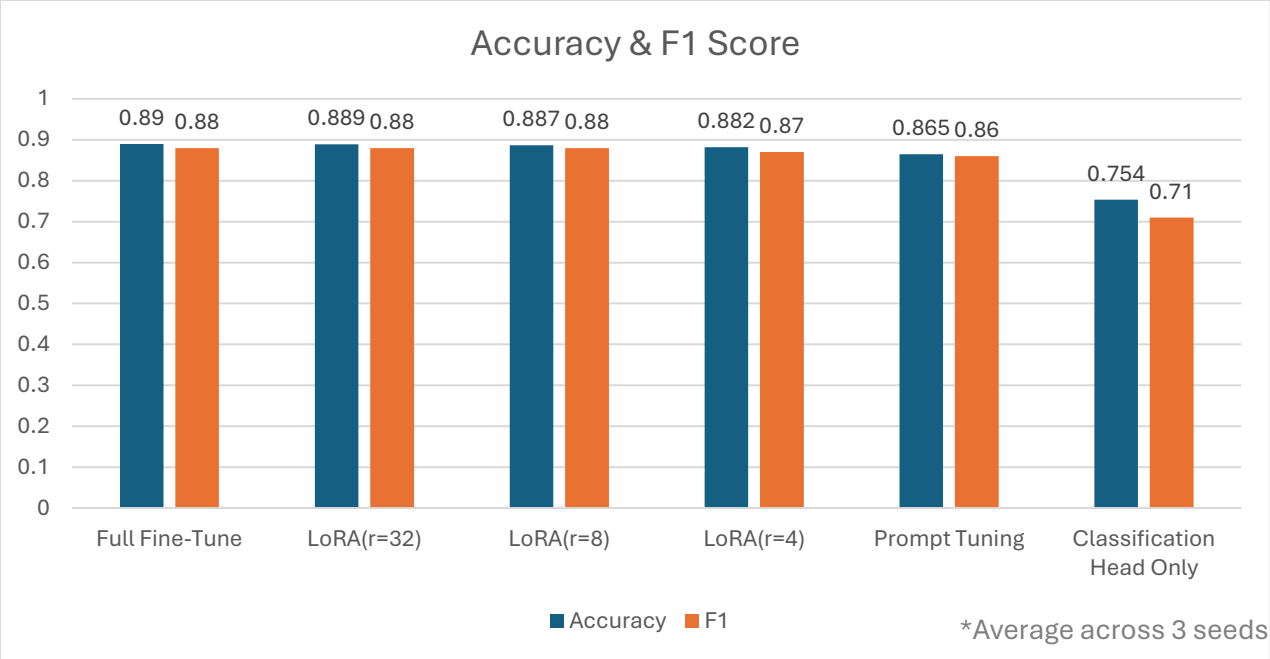
- Learning rate: $5e-4$
- Sequence length reduced to accommodate prompt tokens

5. Results

5.1 Training Time (Average across 3 runs)



5.2 Accuracy and F1 Score (Average across 3 seeds)



6. Discussion

6.1 Efficiency vs Performance

- Full fine-tuning achieves the best results but requires the most training time
- LoRA achieves near-identical performance with ~22% less training time
- Prompt tuning is faster but sacrifices accuracy
- Head-only training is fastest but significantly underperforms

6.2 Effect of Trainable Parameters

Performance strongly correlates with how much of the model is allowed to adapt:

- Full tuning → full adaptability → best performance
- LoRA → targeted adaptability → near-optimal
- Prompt tuning → input-level adaptation → moderate performance
- Head-only → minimal adaptation → poor performance

6.3 LoRA Rank Analysis

- Increasing rank improves performance slightly
- Gains plateau beyond $r = 8$

- Indicates that low-rank updates are sufficient for this task

7. Limitations

- Binary classification only
- Limited hyperparameter search
- No detailed error analysis
- Dataset labels contain noise (emoticon-based labeling)

8. Future Work

- Extend to multi-class sentiment (include neutral)
- Explore larger or alternative Transformer models
- Perform error analysis (misclassified examples)
- Generalize to other domains

9. Conclusion

This project compares multiple fine-tuning strategies for sentiment analysis on social media text.

Key findings:

- Full fine-tuning provides the best performance but is computationally expensive
- LoRA achieves nearly identical performance with significantly lower cost
- Prompt tuning offers efficiency but reduced accuracy
- Head-only tuning is insufficient for high-quality performance

Overall, LoRA provides the best balance between efficiency and effectiveness, making it a strong candidate for practical deployment in resource-constrained settings.